Universidad de O'Higgins

ICI INSTITUTO DE CIENCIAS DE LA INGENIERÍA

Procesamiento Masivo de Datos

Nextflow

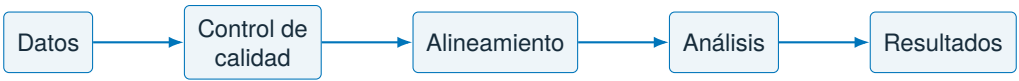
Flujos científicos escalables y reproducibles

Alex Di Genova

Universidad de O'Higgins

De datos genómicos a resultados

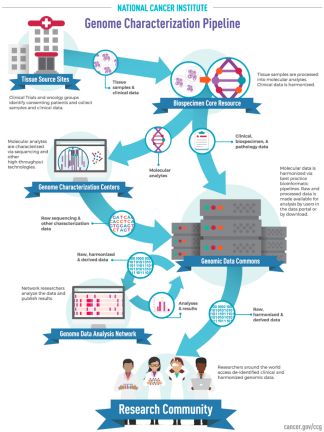
Un flujo de procesos conectados



- **Reproducibilidad:** registrar datos, versiones y configuración.
- **Escalabilidad:** procesar muchas muestras en un clúster o en la nube.
- **Portabilidad:** adaptar la ejecución a distintas infraestructuras.
- **Heterogeneidad:** aislar dependencias y asignar recursos por tarea.

Un análisis genómico como flujo de datos

De la muestra al análisis y almacenamiento

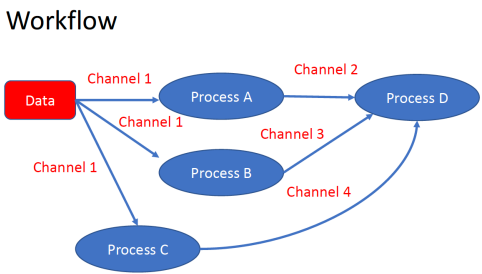


- Las muestras pasan por preparación, caracterización, análisis y almacenamiento.
- Cada etapa produce datos que alimentan a otras tareas.
- Un workflow expresa esas dependencias y coordina el procesamiento de múltiples muestras.

Fuente: National Cancer Institute, Genome Characterization Pipeline.

Procesos conectados por canales

Un flujo puede ramificarse y reunirse



- Los canales transportan valores entre etapas.
- Un proceso puede recibir y producir tuplas con varios componentes.
- Las ramas independientes pueden ejecutarse concurrentemente.

Un lenguaje para describir pipelines

DSL: lenguaje específico de dominio

- Un DSL expresa tareas, entradas, salidas y dependencias del flujo.
- **Nextflow** se basa en Groovy y usa programación orientada al flujo de datos.
- Otras opciones incluyen **Snakemake** y **WDL**.
- El motor programa las tareas cuando sus entradas están disponibles; las tareas independientes pueden ejecutarse en paralelo.

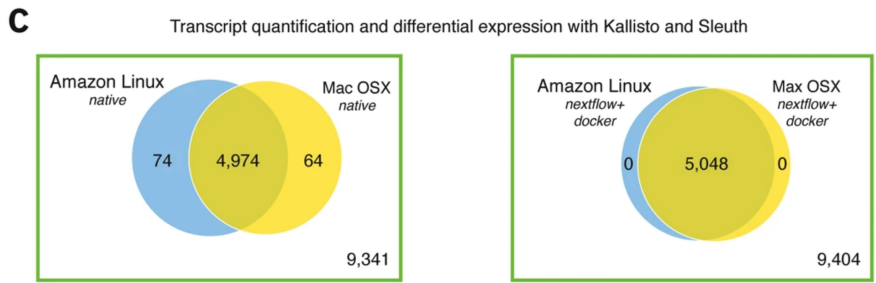
Diseño de un pipeline

Separar análisis, dependencias y ejecución

1. Escribir comandos en Bash, Python, R u otro lenguaje disponible.
2. Definir procesos y conectarlos mediante canales.
3. Especificar dependencias: contenedores o entornos Conda.
4. Configurar recursos y ejecutor: local, planificador HPC o nube.
5. Ejecutar y revisar resultados, registros y reportes.

Reproducibilidad entre plataformas

Kallisto y Sleuth: ejecución nativa y con contenedor

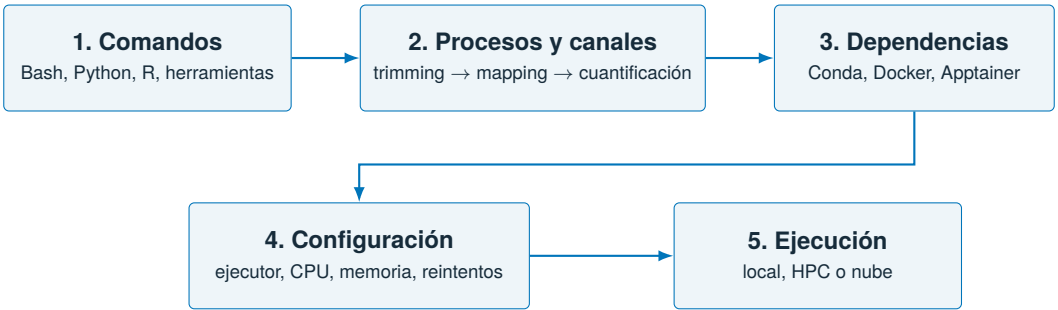


- En ejecución nativa, 4.974 genes diferenciales coincidieron; 74 fueron exclusivos de Linux y 64 de macOS.
- Con Nextflow y Docker, los 5.048 genes detectados coincidieron entre ambos entornos en este experimento.

Di Tommaso et al., Nature Biotechnology, 2017, Fig. 1c. La reproducibilidad también depende de los datos, parámetros y versiones.

Diseño de un programa Nextflow

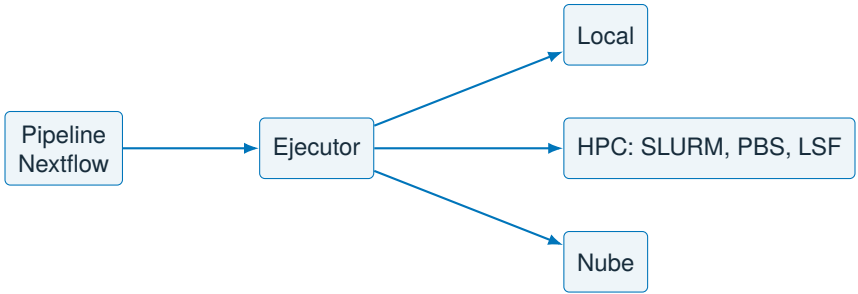
Del comando científico al flujo ejecutable



Las tareas se activan cuando sus entradas están disponibles y las tareas independientes pueden ejecutarse en paralelo.

Un flujo, distintas infraestructuras

El ejecutor traduce tareas al entorno disponible



- El acceso a datos, los recursos y las dependencias deben configurarse para cada entorno.
- Un clúster combina nodos de cómputo, red y un planificador de trabajos.

Dependencias y contenedores

Empaquetar el entorno de cada herramienta

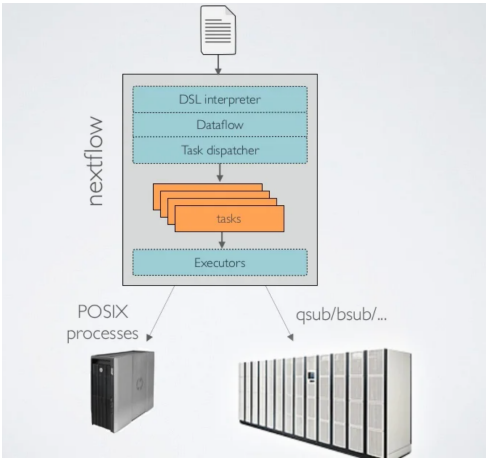
- Una imagen reúne herramientas y bibliotecas con versiones definidas.
- Docker y Apptainer/Singularity permiten usar imágenes en entornos compatibles.
- Conda ofrece otra forma de gestionar dependencias.
- El ejecutor y el sistema de contenedores se configuran por separado.

Para reproducir un análisis

Fijar versión del pipeline, imágenes y parámetros; conservar las versiones de los datos de entrada.

Arquitectura de ejecución

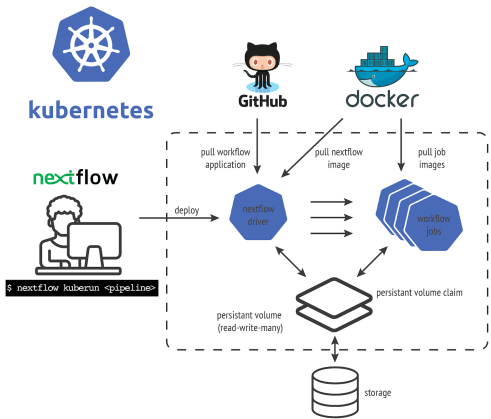
Del DSL a las tareas



- Nextflow interpreta el flujo y determina las tareas listas.
- El ejecutor las inicia localmente o las envía al planificador.
- Los procesos y sus comandos se ejecutan en el entorno configurado.

Integración con infraestructura

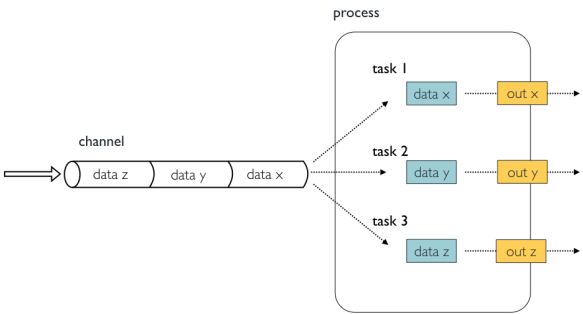
Ejemplo conceptual de despliegue



- El flujo, las imágenes de software y los datos se gestionan como componentes distintos.
- El entorno de cómputo recibe las tareas y accede al almacenamiento.
- La configuración concreta depende de la plataforma y del almacenamiento disponible.

Un elemento de canal activa una tarea

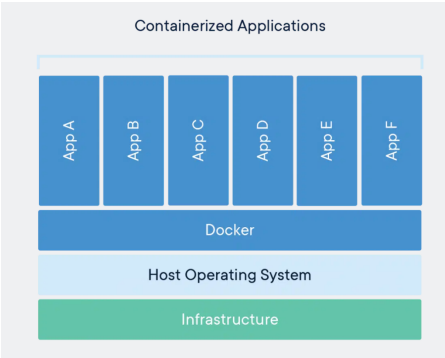
Paralelismo entre muestras



- El proceso recibe elementos del canal y crea instancias de tarea.
- Cada tarea tiene su directorio de trabajo y produce sus propias salidas.
- El planificador puede ejecutar tareas listas simultáneamente según los recursos disponibles.

Imágenes y entornos de ejecución

Dependencias reproducibles

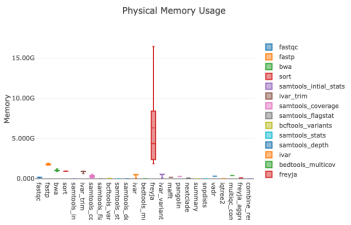




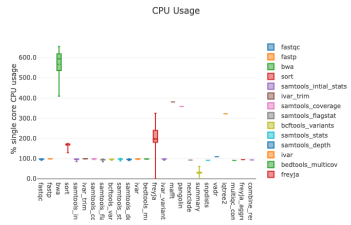
- Cada proceso puede usar una imagen con las versiones de herramientas y bibliotecas que necesita.
- Fijar una etiqueta inmutable o un digest ayuda a reconstruir el entorno.

Interpretar métricas por proceso

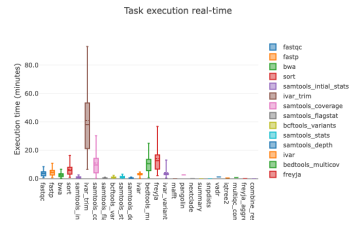
Detectar cuellos de botella



Memoria física



Uso de CPU



Tiempo por tarea

- Comparar tareas y procesos permite ajustar memoria, CPU y tiempo solicitados.
- Las métricas disponibles dependen del ejecutor y del entorno de ejecución.

De la pesadilla de dependencias a un flujo reproducible

Cada herramienta necesita su propio entorno

Un análisis puede necesitar

FastQC, Trim Galore, STAR, SAMtools, GATK, MultiQC, R y bibliotecas específicas.

- Versiones incompatibles y múltiples instalaciones complican la ejecución.

Con Nextflow y contenedores

El workflow declara las tareas; las imágenes aportan las herramientas y dependencias que cada una necesita.

```
nextflow run org/pipeline \
  -r v1.0 -profile singularity \
  --input samplesheet.csv
```

Ejemplo esquemático: el perfil y los parámetros dependen del pipeline; el motor de contenedores debe estar disponible.

Ejecutar un pipeline

Repositorio, revisión y perfil

```
nextflow run digenoma-lab/longreadstats \
    -r v1.0dev -profile singularity \
    --input samplesheet.csv
```

- `digenoma-lab/longreadstats`: repositorio del pipeline.
- `-r`: revisión, etiqueta o rama; fijar una versión favorece la reproducibilidad.
- `-profile`: configuración proporcionada por el pipeline.
- `-input`: parámetro del pipeline; su formato depende del proyecto.

Consultar el repositorio para verificar revisiones, perfiles y parámetros disponibles.

Configuración y reanudación

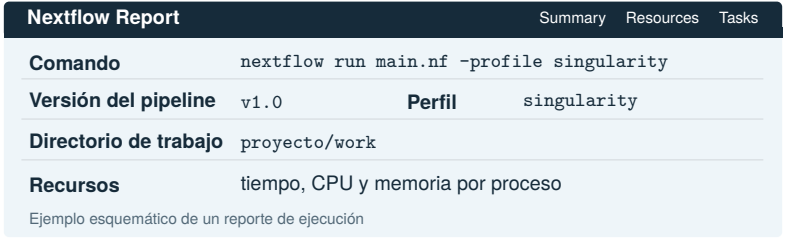
Adaptar recursos sin modificar el análisis

```
profiles {
    local { process.executor = 'local' }
    slurm { process.executor = 'slurm' }
}
process {
    cpus = 2
    memory = '4 GB'
    time = '2h'
}
```

- Seleccionar un perfil con `-profile slurm`.
- Cada proceso puede definir sus propios recursos.
- `-resume` reutiliza tareas en caché cuando es posible; conservar el directorio `work` y la caché.

Reportes de ejecución

Resumen y trazabilidad del workflow



Ejemplo esquemático de un reporte de ejecución

```
nextflow run main.nf -with-report reporte.html
```

Procesos y canales

La unidad de ejecución es una tarea

- Un **proceso** declara entradas, salidas y el comando que se ejecuta.
- Un **canal** transporta valores o rutas entre procesos.
- Cada elemento de entrada puede generar una tarea; las tareas trabajan en directorios aislados.
- El flujo conecta procesos explícitamente; el motor resuelve dependencias y planifica tareas.

Primer proceso

Contar líneas de archivos FASTQ comprimidos

```
process CONTAR_LINEAS {
    tag "${read.name}"
    input:
    path read
    output:
    tuple val(read.name), path('lineas.txt')
    script:
    """
    gzip -cd '${read}' | wc -l > lineas.txt
    """
}
```

Cada tarea escribe su propio `lineas.txt`. El resultado es una tupla con nombre y archivo.

Conectar el proceso

Crear un canal y observar la salida

```
params.reads = 'data/*.fq.gz'
workflow {
    reads = Channel.fromPath(params.reads, checkIfExists: true)
    CONTAR_LINEAS(reads).view { nombre, archivo ->
        "${nombre}: ${archivo.text.trim()} lineas"
    }
}
```

```
nextflow run examples/conteo.nf --reads 'data/*.fq.gz'
```

Archivo completo: `examples/conteo.nf`. En FASTQ válido, cuatro líneas representan una lectura; el conteo es de líneas.

Un flujo con dos etapas

Índice compartido y cuantificación por muestra

```
workflow {
    referencia = Channel.fromPath(params.ref, checkIfExists: true)
    pares = Channel.fromFilePairs(params.reads, checkIfExists: true)
    indice = INDEX(referencia.first())
    QUANT(indice, pares)
}
```

- `INDEX` produce un índice; `QUANT` usa ese índice con cada par de lecturas.
- `first()` produce un valor reutilizable: el ejemplo supone exactamente una referencia.
- Los procesos declaran sus entradas y salidas; a continuación se presenta el ejemplo con Salmon.

Ejemplo Salmon: construir el índice

Una referencia de transcriptoma

```
process INDEX {
    cpus 2
    input:
    path transcriptome
    output:
    path 'indice'
    script:
    """
    salmon index -t '${transcriptome}' -i indice -p ${task.cpus}
    """
}
```

El proceso recibe un archivo de referencia y produce un directorio con el índice. Código completo: `examples/salmon.nf`.

Ejemplo Salmon: cuantificar cada muestra

Una tarea por par de lecturas

```
process QUANT {
    cpus 2
    input:
    path indice
    tuple val(sample), path(reads)
    output:
    tuple val(sample), path('quant')
    script:
    """
    salmon quant -i '${indice}' -l A -1 '${reads[0]}' \\
        -2 '${reads[1]}' -p ${task.cpus} -o quant
    """
}
```

La salida conserva el ID junto al directorio de resultados. El ejemplo requiere Salmon y datos compatibles.

Crear canales

Valores y archivos

```
Channel.of('A', 'B', 'C')       // tres elementos
Channel.of(['A', 'B', 'C'])     // un elemento: una lista
Channel.fromPath('data/*.fq.gz', checkIfExists: true)
Channel.fromFilePairs('data/*_{1,2}.fq.gz', checkIfExists: true)
```

- `val` entrega un valor; `path` prepara archivos para la tarea.
- `fromFilePairs` emite `[id, [read1, read2]]` por muestra.
- Las comillas en Bash evitan que el shell expanda el patrón antes de Nextflow.

Un proceso por par de lecturas

Mantener el identificador junto a sus archivos

```
process MOSTRAR_PARES {
    input:
    tuple val(sample), path(reads)
    output:
    stdout
    script:
    """
    printf '%s: %s %s\n' '${sample}' '${reads[0]}' '${reads[1]}'
    """
}
workflow {
    pares = Channel.fromFilePairs(params.reads, checkIfExists: true)
    MOSTRAR_PARES(pares).view()
}
```

Código completo: `examples/pares.nf`. La tarea solo muestra nombres; sustituir el comando por la herramienta de interés.

Una tabla de muestras

Construir tuplas a partir de CSV

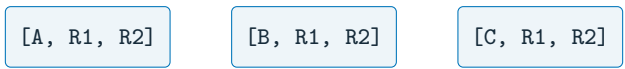
```
sampleId,read1,read2
muestraA,data/muestraA_1.fq.gz,data/muestraA_2.fq.gz
muestraB,data/muestraB_1.fq.gz,data/muestraB_2.fq.gz
```

```
muestras = Channel.fromPath(params.sheet, checkIfExists: true)
    .splitCsv(header: true)
    .map { row -> tuple(row.sampleId, file(row.read1), file(row.read2)) }
MOSTRAR_MUESTRA(muestras)
```

Código completo: `examples/muestras.nf`. Las rutas relativas se interpretan desde el directorio de ejecución.

Anatomía de un canal

Elementos, tuplas y cardinalidad



Un canal con tres elementos
cada elemento es una tupla de tres campos

- Mantener ID y archivos en una sola tupla conserva la asociación entre ellos.
- Una lista dentro de un elemento es distinta de varios elementos independientes.

Crear canales: `of`, `value` y `fromPath`

Cuántos elementos se emiten

```
channel.of('A', 'B', 'C').view()        // A  B  C: tres elementos
channel.of(['A', 'B', 'C']).view()      // [A, B, C]: un elemento
channel.value('ref.fa')                 // un valor reutilizable
channel.fromPath('data/*.fq.gz', checkIfExists: true).view()
```

- `of` emite cada argumento; `value` crea un canal de valor.
- `fromPath` busca archivos que correspondan al patrón y emite rutas.
- Un canal de valor puede reutilizarse para cada elemento de un canal de cola.

Leer pares y tablas

`fromFilePairs` y `splitCsv`

```
channel.fromFilePairs('data/*_{1,2}.fq.gz').view()
// [muestraA, [muestraA_1.fq.gz, muestraA_2.fq.gz]]

channel.fromPath('examples/muestras.csv')
    .splitCsv(header: true)
    .map { row -> tuple(row.sampleId, file(row.read1), file(row.read2)) }
    .view()
// [muestraA, ruta/a/muestraA_1.fq.gz, ruta/a/muestraA_2.fq.gz]
```

`flat: true` en `fromFilePairs` emite ID y archivos en una tupla plana. Las rutas mostradas son esquemáticas.

Transformar y filtrar

`map`, `filter` y closures

```
channel.of(['A', 12], ['B', 5], ['C', 20])
    .filter { id, n -> n >= 10 }
    .map { id, n -> tuple(id, n * 2) }
    .view()
// [A, 24]
// [C, 40]
```

- `filter` conserva elementos que cumplen el predicado.
- `map` transforma cada elemento; `it` representa el argumento cuando se omite su nombre.
- `view` permite inspeccionar el contenido del canal.

Reordenar tuplas con `map`

Cambiar forma sin perder asociación

```
channel.of(['A', 'A_1.fq', 'A_2.fq'],
           ['B', 'B_1.fq', 'B_2.fq'])
    .map { id, r1, r2 -> tuple(id, [r1, r2]) }
    .view()
// [A, [A_1.fq, A_2.fq]]
// [B, [B_1.fq, B_2.fq]]
```

El resultado tiene dos campos: un ID y una lista con dos nombres. Para entradas `path`, usar objetos archivo en lugar de cadenas.

`flatten`: separar listas anidadas

Más elementos, menos estructura

```
channel.of(['A', ['A_1.fq', 'A_2.fq']])
    .flatten()
    .view()
// A
// A_1.fq
// A_2.fq
```

- `flatten` desanida recursivamente las listas y emite sus valores.
- Puede destruir la asociación entre muestra y archivos: usarlo solo si esa estructura ya no es necesaria.

`collect` frente a `toList`

Un solo elemento de salida, distinta estructura

```
channel.of(['A', 1], ['B', 2]).collect().view()
// [A, 1, B, 2]
channel.of(['A', 1], ['B', 2]).toList().view()
// [[A, 1], [B, 2]]
channel.of(['A', 1], ['B', 2]).collect(flat: false).view()
// [[A, 1], [B, 2]]
```

Ambos esperan el cierre. En entrada vacía, `collect()` no emite y `toList()` emite `[]`. El orden depende del canal de entrada.

Ordenar o concatenar

`toSortedList` y `concat`

```
channel.of('C', 'A', 'B').toSortedList().view()
// [A, B, C]

channel.of('A', 'B').concat(channel.of('C', 'D')).view()
// A, B, C, D
```

- `toSortedList` reúne todos los elementos y entrega una lista ordenada.
- `concat` consume un canal después del otro; no forma pares ni producto cartesiano.
- Ordenar puede ser necesario antes de comparar resultados de tareas concurrentes.

`combine`: todas las combinaciones

Muestras por regiones genómicas

```
muestras = channel.of('A', 'B')
regiones = channel.of('chr1', 'chr2')
muestras.combine(regiones).view()
// [A, chr1]  [A, chr2]  [B, chr1]  [B, chr2]
```

- Sin clave, produce el producto cartesiano: dos muestras por dos regiones generan cuatro elementos.
- Con `by: 0`, combina tuplas que comparten la clave de la posición 0; admite múltiples coincidencias por clave.

`join`: asociar por identificador

Un archivo BAM con su índice

```
bam = channel.of(['A', 'A.bam'], ['B', 'B.bam'])
bai = channel.of(['A', 'A.bai'], ['B', 'B.bai'])
bam.join(bai, failOnDuplicate: true).view()
// [A, A.bam, A.bai]
// [B, B.bam, B.bai]
```

Por defecto usa la primera posición y descarta claves sin pareja. `join` no implementa el producto cartesiano de claves duplicadas; para ello usar `combine(by: 0)`.

`groupTuple`: reunir por clave

Combinar lanes de una muestra

```
channel.of(['A', 'A_L1.fq'], ['B', 'B_L1.fq'],
           ['A', 'A_L2.fq'])
    .groupTuple()
    .view()
// [A, [A_L1.fq, A_L2.fq]]
// [B, [B_L1.fq]]
```

- Agrupa los valores de las tuplas según su clave, por defecto el primer campo.
- Sin tamaño conocido, espera el cierre del canal; `size: n` permite emitir grupos completos antes.

Funciones y closures de Groovy

Transformaciones reutilizables

```
def etiqueta(id, n) {
    return "${id}: ${n} lecturas"
}
workflow {
    channel.of(['A', 100], ['B', 75])
        .map { id, n -> etiqueta(id, n) }
        .view()
}
// A: 100 lecturas
// B: 75 lecturas
```

Una función retorna un valor; un proceso declara una tarea ejecutable. La closure de `map` se aplica a cada elemento.

Leer una lista de archivos

`splitText` y conversión a rutas

```
// archivos.txt contiene una ruta por linea
channel.fromPath('examples/archivos.txt', checkIfExists: true)
    .splitText()
    .map { linea -> linea.trim() }
    .filter { linea -> linea && !linea.startsWith('#') }
    .map { ruta -> file(ruta, checkIfExists: true) }
    .view()
// /.../data/muestraA_1.fq.gz
// /.../data/muestraB_1.fq.gz
```

`splitText()` entrega una línea por elemento; `trim()` quita espacios y salto de línea. Las rutas relativas se resuelven desde el directorio de ejecución.

Leer una tabla TSV

Seleccionar columnas y añadir una ruta base

```
params.bam_dir = 'data/bam'
channel.fromPath('muestras.tsv', checkIfExists: true)
    .splitCsv(header: true, sep: '\t')
    .map { row ->
        def bam = file("${params.bam_dir}/${row.bam}")
        tuple(row.sample, bam, file("${bam}.bai"))
    }
    .view()
// [A, /.../data/bam/A.bam, /.../data/bam/A.bam.bai]
```

Ejemplo con columnas `sample` y `bam`, separadas por tabulación: se conserva el identificador junto al BAM y su índice. No comprueba aquí la existencia de ambos archivos.

`flatMap`: expandir sin perder la clave

Una tarea por región de cada muestra

```
channel.of(['A', ['chr1', 'chr2']], ['B', ['chr1']])
    .flatMap { id, regiones ->
        regiones.collect { chr -> tuple(id, chr) }
    }
    .view()
// [A, chr1]  [A, chr2]  [B, chr1]
```

- La closure retorna una lista de tuplas; `flatMap` emite cada tupla.
- Se conserva el ID para reunir las salidas más adelante con `groupTuple` o `join`.

Expandir una tupla agrupada

`transpose` y alternativa explícita

```
channel.of(['A', ['L1.fq', 'L2.fq']])
    .transpose().view()
// [A, L1.fq]  [A, L2.fq]

channel.of(['A', ['L1.fq', 'L2.fq']])
    .flatMap { id, lanes -> lanes.collect { f -> tuple(id, f) } }
    .view()
```

Para este caso ambas formas preservan el ID. Con varias listas, `transpose` combina sus posiciones; consultar `remainder` para longitudes diferentes.

Proteger una lista al combinar canales

Agregar una capa con `map`

```
vcfs = channel.of(['A.vcf', 'A.vcf.tbi'])
ref = channel.value('contigs.txt')
vcfs.combine(ref).view()
// [A.vcf, A.vcf.tbi, contigs.txt]

vcfs.map { par -> [par] }.combine(ref).view()
// [[A.vcf, A.vcf.tbi], contigs.txt]
```

- `combine` concatena los campos de listas y tuplas.
- Al encapsular el par, este permanece como un único campo; sirve para declarar una entrada `tuple path(vcfs), path(ref)`.

`combine(by: 0)` con claves duplicadas

Todas las coincidencias por muestra

```
izq = channel.of(['A', 'bam1'], ['A', 'bam2'], ['B', 'bam3'])
der = channel.of(['A', 'idx1'], ['A', 'idx2'])
izq.combine(der, by: 0).view()
// [A, bam1, idx1]  [A, bam1, idx2]
// [A, bam2, idx1]  [A, bam2, idx2]
```

Se comparan las claves de posición 0 y se generan todas sus combinaciones; B no tiene pareja. Para claves únicas usar `join` con comprobación de duplicados.

`join`: detectar entradas sin pareja

Validar antes de procesar

```
bam = channel.of(['A', 'A.bam'], ['B', 'B.bam'])
bai = channel.of(['A', 'A.bai'])
bam.join(bai, remainder: true, failOnDuplicate: true).view()
// [A, A.bam, A.bai]
// [B, B.bam, null]
```

- Con `remainder: true`, las claves sin pareja se emiten con `null`.
- Para fallar ante una clave sin pareja se puede usar `failOnMismatch: true`, sin `remainder`.

`mix` frente a `concat`

Reunir elementos sin asociarlos

```
a = channel.of('A1', 'A2')
b = channel.of('B1', 'B2')
a.concat(b).view() // A1, A2, B1, B2
// mix permite intercalar las emisiones:
a.mix(b).view()    // por ejemplo B1, A1, A2, B2
```

- `concat` espera a que termine cada canal antes de consumir el siguiente.
- `mix` reúne elementos conforme llegan; el orden entre canales es indeterminado. Ninguno une muestras por ID.

`branch`: enrutar por condición

Una muestra se asigna a una rama

```
channel.of(['A', 85], ['B', 40], ['C', 70])
    .branch { id, calidad ->
        aprobado: calidad >= 70
        revision: true
    }.set { qc }
qc.aprobado.view()  // [A, 85], [C, 70]
qc.revision.view()  // [B, 40]
```

Cada elemento va a la primera condición verdadera. Las ramas se ejecutan independientemente y el orden de impresión puede variar.

`multiMap`: crear dos vistas

Conservar la clave en ambas ramas

```
channel.of(['A', 'A.bam'], ['B', 'B.bam'])
    .multiMap { id, bam ->
        calidad: tuple(id, bam)
        indice:  tuple(id, "${bam}.bai")
    }.set { salidas }
salidas.calidad.view() // [A, A.bam], [B, B.bam]
salidas.indice.view()  // [A, A.bam.bai], [B, B.bam.bai]
```

Cada elemento origina un valor en cada rama. Mantener el ID permite asociar luego las salidas mediante `join`.

Eliminar repeticiones

`unique` y `distinct`

```
channel.of('A', 'A', 'B', 'A').unique().view()
// A, B
channel.of('A', 'A', 'B', 'A').distinct().view()
// A, B, A
```

- `unique` elimina valores duplicados en todo el canal.
- `distinct` elimina solo repeticiones consecutivas: su resultado depende del orden de llegada.

Resumir y validar

`count`, `reduce` e `ifEmpty`

```
channel.of(10, 20, 30).count().view() // 3
channel.of(10, 20, 30).reduce(0) { acc, n -> acc + n }.view()
// 60
channel.empty().ifEmpty('sin muestras').view()
// sin muestras
```

`count` y `reduce` producen un valor al finalizar el canal. `ifEmpty` permite manejar explícitamente una entrada sin elementos.

`collectFile`: crear un resumen

Una salida a partir de varios elementos

```
channel.of('A\t12', 'B\t8')
    .collectFile(name: 'resumen.tsv', newLine: true)
    .view { archivo -> archivo.text }
// A    12
// B     8
```

- Reúne contenido y emite la ruta del archivo creado.
- Para conservar el resultado fuera del trabajo temporal, publicarlo explícitamente desde el workflow o proceso correspondiente.

Cómo elegir una operación

Estructura y cardinalidad de salida

Objetivo	Operador	Resultado
Transformar un elemento	`map`, `filter`	cero o un elemento por entrada
Expandir por muestra	`flatMap`	varios elementos con su clave
Unir por ID	`join`, `combine(by: 0)`	coincidencias por clave
Reunir por ID	`groupTuple`	una tupla por grupo
Reunir todo	`toList`, `collect`	un valor al cerrar el canal
Separar el flujo	`branch`, `multiMap`	varios canales de salida

Ejercicio 1: filtrar y transformar

Predecir la salida antes de ejecutar

```
channel.of(['A', 12], ['B', 5], ['C', 20])
    .filter { id, n -> n >= 10 }
    .map { id, n -> tuple(id, n * 2) }
    .view()
```

1. ¿Cuántos elementos emite el canal final?
2. ¿Cuál es la estructura y el contenido de cada elemento?
3. ¿Qué cambia si el umbral se reemplaza por `n > 12`?

Solución 1: `filter` y `map`

Dos tuplas pasan el filtro

```
// Salida del filtro: [A, 12], [C, 20]
// Salida de map:     [A, 24], [C, 40]
```

- `filter` descarta la muestra B; `map` duplica el número y conserva el ID.
- El canal final emite dos elementos; cada uno es una tupla `[id, n]`.
- Con `n > 12`, solo se emite `[C, 40]`.

Ejercicio 2: unir por muestra

Archivos BAM y sus índices

```
bam = channel.of(['A', 'A.bam'], ['B', 'B.bam'])
bai = channel.of(['B', 'B.bai'], ['A', 'A.bai'])
bam.join(bai).view()
```

1. ¿Qué tuplas se emiten? ¿Importa el orden en que llegan los archivos?
2. ¿Qué ocurre si falta el índice de la muestra B?
3. ¿Qué opción permite observar las muestras sin pareja?

Solución 2: `join`

La primera columna es la clave

```
// [A, A.bam, A.bai]
// [B, B.bam, B.bai]
bam.join(bai, remainder: true).view()
// Si falta B.bai: [B, B.bam, null]
```

- La asociación usa el ID, no la posición de llegada; el orden de salida puede variar.
- Por defecto se descartan claves sin pareja; `remainder: true` permite observarlas.

Ejercicio 3: agrupar lanes

Una salida por muestra

```
channel.of(['A', 'L1'], ['B', 'L1'], ['A', 'L2'])
    .groupTuple()
    .view()
```

1. ¿Cuántas tuplas produce el operador?
2. ¿Cuál es el contenido de cada tupla?
3. ¿Cuándo puede emitir los grupos si no se especifica su tamaño?

Solución 3: `groupTuple`

Agrupación por la primera columna

```
// [A, [L1, L2]]
// [B, [L1]]
```

- Se emite una tupla por clave, con una lista de los valores asociados.
- Sin un tamaño de grupo conocido, `groupTuple()` espera el cierre del canal para emitir.
- El orden de llegada puede variar si los elementos provienen de tareas concurrentes.

Ejercicio 4: reunir sin perder estructura

Comparar `collect` y `toList`

```
channel.of(['A', 1], ['B', 2]).collect().view()
channel.of(['A', 1], ['B', 2]).toList().view()
```

1. ¿Qué lista produce cada operador?
2. ¿Cuál conserva cada tupla como un elemento de la lista?
3. ¿Qué ocurre con cada operador cuando el canal está vacío?

Solución 4: `collect` y `toList`

Diferencias de estructura y entrada vacía

```
// collect(): [A, 1, B, 2]
// toList():  [[A, 1], [B, 2]]
```

- `collect()` aplana un nivel por defecto; `toList()` conserva las tuplas.
- Si el canal está vacío, `collect()` no emite; `toList()` emite `[]`.
- Para evitar aplanar con `collect`, usar `collect(flat: false)`.

Reunir salidas para una tarea final

Un archivo por tarea; un resumen para todos

```
workflow {
    reads = Channel.fromPath(params.reads, checkIfExists: true)
    archivos = DESCOMPRIMIR(reads)
    RESUMIR(archivos.collect()).view()
}
```

- `DESCOMPRIMIR` ejecuta una tarea por archivo.
- `collect()` espera las salidas y las entrega como una lista.
- `RESUMIR` recibe los archivos reunidos y ejecuta una tarea.

Implementación completa: `examples/agrupar.nf`. Se esperan archivos con extensión `.fq.gz`.

Una tarea falló: ¿dónde comenzar?

Identificar el proceso y su directorio de trabajo

```
ERROR ~ Error executing process > 'FASTQC (muestraA)'
Caused by:
  Process `FASTQC (muestraA)` terminated with an error exit status
Work dir:
  /proyecto/work/ab/1234567890abcdef
```

1. Leer el nombre de la tarea, la causa y la ruta `Work dir` en el mensaje.
2. Inspeccionar el directorio de esa tarea y el registro general `.nextflow.log`.
3. Corregir el problema y repetir la ejecución; usar `-resume` cuando corresponda.

La ruta mostrada es ilustrativa; usar la que imprime la ejecución.

Archivos de una tarea

El directorio `work` conserva evidencia

```
cd /ruta/indicada/en/Work_dir
ls -la
cat .command.sh       # comando generado para la tarea
cat .command.err      # salida de error estandar
cat .command.out      # salida estandar
cat .exitcode         # codigo de salida, si esta disponible
```

- Comparar el comando generado, las rutas y los archivos creados con lo declarado en `input` y `output`.
- Revisar también `.nextflow.log` para detalles de la ejecución.

Errores frecuentes y comprobaciones

Diagnosticar antes de volver a ejecutar

Síntoma	Qué revisar
Comando no encontrado	Dependencias, contenedor y perfil de ejecución.
Archivo de salida ausente	Nombre, ruta y patrón declarados en `output`.
Entrada o tupla incorrecta	Contenido de canales con `view()` y tipos `val`/`path`.
Tarea terminada por señal	Límites de memoria/tiempo y registros del planificador.

```
nextflow run main.nf -resume
```

`-resume` reutiliza tareas válidas de la caché cuando es posible; conservar `work/` y `.nextflow/`.

Organizar un proyecto
Módulos y flujos reutilizables

```
include { FASTQC } from './modules/fastqc'
include { BWAMEM } from './modules/bwamem'
include { SAMTOOLS } from './modules/samtools'

workflow {
    // Construir canales y conectar los procesos necesarios.
}
```

- Los módulos separan procesos y facilitan su reutilización.
- Ejemplo del curso: digenoma-lab/BRCA.
- Nextflow utiliza expresiones de Groovy para transformar valores y construir flujos.

¿Qué es nf-core?

Comunidad y ecosistema de pipelines bioinformáticos

- Comunidad que desarrolla y mantiene pipelines de Nextflow siguiendo buenas prácticas compartidas.
- Ofrece **pipelines**, **módulos** reutilizables, plantillas y herramientas para desarrollo.
- Sus convenciones abarcan documentación, pruebas, control de versiones y dependencias.
- Un pipeline creado con la plantilla no pasa automáticamente a ser un pipeline oficial de nf-core.

Explorar pipelines y guías: nf-co.re y documentación nf-core.

¿Cuánto hay en nf-core?

Explorar antes de implementar

Pipelines completos

160

pipelines en el catálogo de nf-core

Ejemplos: RNA-seq, variantes, metagenómica y ensamblaje.

Componentes reutilizables

2.141

módulos para herramientas y tareas

Ejemplos: FastQC, SAMtools, alineadores y cuantificación.

- Buscar primero un pipeline completo; si no existe, reutilizar módulos para construir el flujo.

Catálogos consultados el 9/11/2026: pipelines y módulos. Son recuentos del catálogo, no solo versiones publicadas; pueden cambiar.

Crear un pipeline con nf-core/tools

Plantilla para comenzar un proyecto

```
nf-core pipelines create -n longreadstats \
  -d "Estadisticas de lecturas largas" \
  -a "Alex Di Genova"
```

- La plantilla crea la estructura inicial del proyecto y su repositorio Git.
- Después se implementan los procesos, se conectan los canales y se agregan pruebas.
- Consultar `nf-core pipelines create -help`: las opciones dependen de la versión de nf-core/tools.

Guía: crear un pipeline.

Módulos nf-core

Descubrir y reutilizar procesos

```
nf-core modules list remote fastqc
nf-core modules install fastqc
nf-core modules list local
```

- Ejecutar la instalación desde el directorio del pipeline; se incorporan los archivos del módulo.
- Agregar la sentencia `include` indicada por la herramienta y conectar sus entradas y salidas en el workflow.
- Los módulos incluyen definiciones y metadatos para facilitar su reutilización y mantenimiento.

Guía: añadir módulos a un pipeline.

Programar Nextflow con asistencia de IA

Conectar, simular y revisar cada herramienta

1. Buscar módulos en nf-core y abrir el proyecto con un asistente, por ejemplo OpenCode.
2. Pedir ayuda para conectar entradas y salidas (*wiring*) y preparar bloques `stub`.
3. Revisar el código generado y, para cada herramienta, sus parámetros, versiones, recursos y formatos de entrada y salida.

Ejemplo de instrucción

«Integra nf-core/fastqc, conserva el ID de muestra y prepara una prueba del flujo con datos mínimos y stubs.»

```
opencode
nextflow run main.nf -stub-run
```

Con bloques `stub`, la corrida puede comprobar conexiones y forma de las salidas sin ejecutar los comandos reales. Luego validar cada herramienta con datos pequeños y resultados esperados.

Del desarrollo asistido a una corrida en clúster

Versionar, documentar y probar antes de escalar

1. **Git y GitHub:** guardar el pipeline, la configuración y las pruebas en un repositorio; revisar los cambios generados por IA antes de incorporarlos.
2. **Documentar:** describir el objetivo, las entradas, los parámetros por herramienta, las versiones, las dependencias y cómo reproducir la ejecución.
3. **Buscar datos pequeños:** utilizar datos de prueba apropiados para el problema y con resultados conocidos o verificables.
4. **Validar:** ejecutar primero con stubs y después con las herramientas reales sobre esos datos; revisar salidas, registros y recursos.
5. **Escalar:** fijar la versión del proyecto y lanzar en el clúster con el perfil y los recursos adecuados.

Documentación y referencias

Herramientas para seguir aprendiendo

- Documentación de Nextflow: lenguaje, procesos y configuración.
- Operadores de canales: opciones y ejemplos.
- Nextflow cheatsheet: guía rápida de sintaxis y patrones.
- Documentación de Apache Groovy: lenguaje y biblioteca.
- Documentación de nf-core: pipelines, módulos y herramientas.

Práctica en Google Colab

Abrir los notebooks del curso

Abrir Nextflow I en Colab

Introducción a Nextflow, primer proceso y conteo de líneas.

Abrir Hadoop y Nextflow en Colab

Ejercicios de Hadoop y ejemplos de ejecución con Nextflow.

Los notebooks instalan dependencias en el entorno de Colab; las versiones y la disponibilidad de sus servicios pueden cambiar.

Preguntas?